Database Dumps & MongoDB Backup on AWS

Concepts, commands, security, restore, S3 storage and troubleshooting

Purpose: Explain database dumps and related backup concepts, then provide practical examples for taking a MongoDB dump from an AWS-hosted database (such as MongoDB on EC2 or in Docker) and optionally storing it in Amazon S3.

1. What is a database dump?

A database dump is an export of database contents into files that can later be used to restore or migrate the data. Depending on the database and tool, it may include records/documents, collection or table definitions, indexes and metadata. A dump is a logical backup: it represents database objects and data rather than a byte-for-byte copy of the underlying disk.

2. Related topics and terminology

- **Backup:** A recoverable copy of data. It may be a logical dump, filesystem/volume snapshot, or managed database backup.
- **Logical backup / dump:** Exports database objects and data through a database-aware tool. MongoDB uses mongodump and mongorestore.
- **Physical backup / snapshot:** Copies storage blocks or database files. Consistency and database-specific procedures still matter.
- **Restore:** Loads backup data into a running database. MongoDB uses mongorestore for mongodump output.
- **Point-in-time recovery (PITR):** Restores to a specific time using a base backup plus a log/oplog stream, where supported.
- **RPO:** Recovery Point Objective: the maximum acceptable data loss measured in time. Example: 15 minutes.
- **RTO:** Recovery Time Objective: the target time to restore service. Example: 2 hours.
- **Retention:** How long backups are kept, such as 7 daily and 4 weekly copies.
- **Encryption:** Protects backup data at rest and in transit using encryption, TLS and controlled access.

3. MongoDB dump basics

mongodump is part of MongoDB Database Tools. It connects to a running MongoDB deployment and writes BSON data and metadata. mongorestore reads that output and restores it to a MongoDB server. Run these utilities in the operating-system terminal, not inside mongosh.

- Directory dump: creates a folder containing BSON and metadata files.
- Archive dump: creates one file, convenient for copying and storage.
- Compressed archive: combines --archive with --gzip to reduce storage use.

mongodump reads data from the live server and can affect performance. For a replica set that continues accepting writes, consider --oplog for a consistent capture, subject to MongoDB's documented limitations. A basic dump should not automatically be treated as a point-in-time backup for every deployment.

4. Typical AWS setup

- MongoDB runs on an EC2 instance, or inside a Docker container on EC2.
- mongodump runs on the EC2 host or a secure administration host that can reach MongoDB.
- The dump is written to a protected local directory.
- Optionally, the dump is uploaded to an S3 bucket using an IAM role or AWS CLI profile.

If MongoDB is MongoDB Atlas, use the Atlas connection string and network access rules. AWS hosting does not automatically mean MongoDB uses AWS IAM for database authentication: MongoDB credentials and AWS credentials are separate unless a specific integration is configured.

5. Install and verify MongoDB Database Tools

Install MongoDB Database Tools using the official MongoDB instructions for your operating system and Ubuntu release. Then verify:

```
mongodump --version
mongorestore --version
```

The package is commonly named mongodb-database-tools when the official repository is configured.

6. Identify how MongoDB is running

```
sudo systemctl status mongod
# If Docker is used:
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
```

If MongoDB is in Docker, the host tool can connect if the port is reachable. Otherwise, run a compatible tools container attached to the same Docker network. Do not expose port 27017 publicly just to take a dump.

7. Prepare a secure backup directory

```
mkdir -p "$HOME/mongodb-backups"
chmod 700 "$HOME/mongodb-backups"
cd "$HOME/mongodb-backups"
BACKUP_DATE=$(date +%Y%m%d_%H%M%S)
echo "$BACKUP_DATE"
```

Run as the intended backup operator and ensure file ownership/permissions allow only authorized users to access the dump.

8. Take a dump from MongoDB on AWS EC2

Option A — MongoDB on the same EC2 instance

Replace the username and database name with your real values. The password is prompted interactively rather than embedded in shell history.

```
mongodump \
  --host 127.0.0.1 \
  --port 27017 \
  --username appadmin \
  --authenticationDatabase admin \
  --password \
  --db YOUR_DATABASE \
```

```
  --archive="$HOME/mongodb-backups/YOUR_DATABASE-$BACKUP_DATE.archive.gz" \
  --gzip
```

When prompted, enter the MongoDB user password. The account needs sufficient read/backup privileges for the selected scope. Change --authenticationDatabase if the user authenticates against a different database.

Option B — Dump all databases

```
mongodump \
  --host 127.0.0.1 --port 27017 \
  --username appadmin \
  --authenticationDatabase admin \
  --password \
  --archive="$HOME/mongodb-backups/all-databases-$BACKUP_DATE.archive.gz" \
  --gzip
```

Omitting --db requests all databases accessible to the account, subject to privileges. mongodump excludes the local database.

Option C — MongoDB in Docker

If port 27017 is published to localhost, use Option A against 127.0.0.1. If not, run mongodump from a tools container attached to the same Docker network, after checking the container and network names.

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
docker network ls
docker inspect mongodb --format "{{json .NetworkSettings.Networks}}"
```

Avoid putting the database password directly in a command string. Prefer an interactive prompt or a protected configuration file supported by your Database Tools version.

Option D — MongoDB Atlas or remote MongoDB

Use the provider's connection string. Ensure DNS, TLS, routing and IP access rules permit the connection. Treat the URI as a secret. If credentials are included in the URI, special characters must be URL-encoded.

```
mongodump \
  --uri="mongodb+srv://DB_USER@YOUR_CLUSTER.example.mongodb.net/?authSource=admin" \
  --password \
  --db YOUR_DATABASE \
  --archive="$HOME/mongodb-backups/YOUR_DATABASE-$BACKUP_DATE.archive.gz" \
  --gzip
```

9. Dump one collection or filtered documents

```
mongodump \
  --host 127.0.0.1 --port 27017 \
  --username appadmin --authenticationDatabase admin --password \
  --db YOUR_DATABASE --collection YOUR_COLLECTION \
  --archive="$HOME/mongodb-backups/collection-$BACKUP_DATE.archive.gz" \
  --gzip
```

Filtered example (requires --collection; adapt the query to your schema):

```
mongodump \
  --host 127.0.0.1 --port 27017 \
  --username appadmin --authenticationDatabase admin --password \
  --db YOUR_DATABASE --collection YOUR_COLLECTION \
  --query '{"status":"archived"}' \
  --archive="$HOME/mongodb-backups/filtered-$BACKUP_DATE.archive.gz" --gzip
```

Filtered dumps are partial backups. Record the exact collection and filter so the file is not mistaken for a full recovery copy.

10. Verify the dump file

```
ls -lh "$HOME/mongodb-backups/"
file "$HOME/mongodb-backups/YOUR_DATABASE-$BACKUP_DATE.archive.gz"
gzip -t "$HOME/mongodb-backups/YOUR_DATABASE-$BACKUP_DATE.archive.gz"
echo "gzip exit code: $?"
```

A successful gzip check verifies the compression stream, not that all documents can be restored. The strongest validation is a test restore into an isolated MongoDB instance followed by document-count and application-level checks.

11. Restore a dump safely

Test restores in an isolated destination first. This example renames the source database namespace to a separate test database; verify the source namespace before use.

```
mongorestore \
  --host 127.0.0.1 --port 27017 \
  --username restore_user --authenticationDatabase admin --password \
  --gzip \
  --archive="$HOME/mongodb-backups/YOUR_DATABASE-$BACKUP_DATE.archive.gz" \
  --nsFrom="YOUR_DATABASE.*" \
  --nsTo="YOUR_DATABASE_restore_test.*"
```

Do not add --drop casually: it drops target collections before restoring. Never test a restore against production without an approved recovery plan and verified target.

12. Upload the dump to Amazon S3

Prefer an EC2 instance profile/IAM role. If using a named AWS CLI profile, verify which identity is active and use least-privilege S3 permissions.

```
aws sts get-caller-identity --profile sandbox
aws s3 ls --profile sandbox
```

Upload an archive to an existing bucket:

```
BUCKET="YOUR-BUCKET-NAME"
FILE="$HOME/mongodb-backups/YOUR_DATABASE-$BACKUP_DATE.archive.gz"
aws s3 cp "$FILE" "s3://$BUCKET/mongodb/$(basename "$FILE")" --profile sandbox
```

Verify the object:

```
aws s3 ls "s3://$BUCKET/mongodb/" --profile sandbox
aws s3api head-object --bucket "$BUCKET" --key "mongodb/$(basename "$FILE")" --profile sandbox
```

Consider S3 server-side encryption, versioning, lifecycle policies and, where appropriate, Object Lock. Never make backup buckets public.

13. Example shell script for a scheduled backup

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077
```

```
BACKUP_DIR="$HOME/mongodb-backups"
DB_NAME="YOUR_DATABASE"
S3_BUCKET="YOUR-BUCKET-NAME"
AWS_PROFILE="sandbox"
STAMP=$(date +%Y%m%d_%H%M%S)
FILE="$BACKUP_DIR/${DB_NAME}-${STAMP}.archive.gz"

mkdir -p "$BACKUP_DIR"

mongodump \
  --host 127.0.0.1 --port 27017 \
  --username appadmin --authenticationDatabase admin --password \
  --db "$DB_NAME" --archive="$FILE" --gzip

gzip -t "$FILE"
aws s3 cp "$FILE" "s3://$S3_BUCKET/mongodb/$(basename "$FILE")" --profile "$AWS_PROFILE"
aws s3api head-object --bucket "$S3_BUCKET" --key "mongodb/$(basename "$FILE")" --profile
"$AWS_PROFILE"
echo "Backup uploaded and verified: $FILE"
```

This is a starting point, not a complete enterprise backup system. Unattended jobs need a secure non-interactive secret method, logging, alerting, retention rules, retry handling and tested restore procedures. Do not store passwords directly in the script.

14. Common problems and troubleshooting

- **mongodump: command not found —** Install MongoDB Database Tools and confirm the binary is on PATH.
- **Authentication failed —** Check username, password, authSource/authenticationDatabase and backup/read privileges.
- **Connection refused to 127.0.0.1:27017 —** MongoDB may not run on the host, may be in Docker without a published port, or may use another port. Check systemctl, docker ps and port configuration.
- **Network timeout —** Check DNS, routes, security groups, NACLs, firewall, bind address, TLS and provider allowlists. Do not open MongoDB to the entire internet.
- **Not authorized on admin —** The user may be able to dump an application database but not admin/system metadata. Use a properly scoped backup account.
- **S3 AccessDenied —** Check aws sts get-caller-identity, profile/role, bucket policy, IAM permissions and KMS permissions if used.
- **Disk full / ENOSPC —** Check df -h and df -i; ensure enough free space and handle pipeline errors/retries if streaming.
- **Restore reports zero documents —** Check exit status/output, source database and collection names, filters, and validate with a real test restore. A dry run does not prove documents were restored.

15. Production checklist

- ☐ Use a dedicated MongoDB backup user with minimum required privileges.
- ☐ Keep MongoDB on private networking; do not broadly expose port 27017.
- ☐ Use TLS for remote database connections and encrypt backup files at rest.
- ☐ Prefer EC2 IAM roles or a tightly controlled AWS profile for S3 access.
- ☐ Set retention and lifecycle rules that meet operational and compliance requirements.
- ☐ Record timestamp, source, database scope, tool version and success/failure.
- ☐ Alert on failed backups and missing expected S3 objects.

- ☐ Test restores regularly in an isolated environment.
- ☐ For replica sets with writes, assess --oplog consistency. For sharded clusters, use a supported coordinated backup strategy.
- ☐ Keep at least one recovery copy protected from accidental or malicious deletion.

16. Security notes

- Never hard-code database passwords, AWS access keys or credential-bearing URIs in scripts, Git, screenshots or tickets.
- AWS IAM credentials do not automatically authenticate MongoDB users; they are separate security systems unless a specific MongoDB Atlas MONGODB-AWS setup is configured.
- Treat dumps as sensitive production data: restrict filesystem permissions, S3 access, sharing and retention.
- Before deleting a backup bucket or object, verify the AWS account/profile, region, bucket name, retention policy and availability of another recovery copy.

17. Official references

- **MongoDB — Back Up and Restore with MongoDB Tools:** https://www.mongodb.com/docs/v8.0/tutorial/backup-and-restore-tools/
- **MongoDB — mongodump reference:** https://www.mongodb.com/docs/database-tools/mongodump/
- **MongoDB — mongodump examples:** https://www.mongodb.com/docs/database-tools/mongodump/mongodump-examples/
- **MongoDB — mongodump behavior, access and usage:** https://www.mongodb.com/docs/database-tools/mongodump/mongodump-behavior/
- **AWS CLI — S3 cp command:** https://docs.aws.amazon.com/cli/latest/reference/s3/cp.html
- **AWS — IAM roles for Amazon EC2:** https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html

End of document